

Assignment: Tech Inject Design Library

Experience: 0–2 years · **AI use required** · **Timebox:** up to eight hours

Scenario: You are a developer tasked with establishing a shared design standard for applications built within your organisation. Every developer in the company will use your component library to build consistent interfaces, and administrators will maintain and expand it over time. Your responsibility is to turn the reference UI into a reusable, documented and dependable system that other engineers can confidently adopt—not just a collection of attractive examples.

1. What to build

Build a reusable Next.js/React + TypeScript component library with **two separately runnable web apps**:

1. **Public component catalogue:** developers browse free and premium components and, where their access allows, use live previews, copied code, installation commands and AI-agent integration prompts.
2. **Admin dashboard:** an administrator uploads/publishes components, labels them free or premium, and manages customers' premium access.

A monorepo with TurboRepo and shared backend are welcome. Choose your backend, persistent database and storage. The admin dashboard can be simple; the public catalogue should feel like a polished developer product.

You are expected to deliver a production-ready application for the scope below—not a prototype. Own the complete implementation, deployment and verification. Use AI extensively throughout planning, development, testing and debugging; AI use is required, not a shortcut to skip understanding. You remain accountable for every change you submit and must answer the written questions in Section 10.

We expect you to fully use AI to build this project. You do not need to write every line manually. What matters is the decisions you make: how you break down the problem, choose between approaches, review AI suggestions and verify the result. Document those decisions in `answers.md`; AI-generated code is welcome, but unexplained or unverified code is not.

2. Visual reference and component scope

Use this as the **visual source of truth for the component theme**:

[Sales CRM reference](#)

Analyse the reference yourself and decide **which UI elements should become reusable components**, how to group them, and which differences belong as variants rather than separate components. We are intentionally not prescribing component types or a count: identifying useful component boundaries is part of the assignment.

In your README, include a short inventory of the components you identified, which you implemented within the timebox, and why you prioritised them. Map implemented components to reference screenshots. Explain important omissions; selection quality, reuse and fidelity matter more than quantity. If the reference is inaccessible, flag it rather than inventing a different theme.

Match the reference closely: typography, colours, borders, radii, spacing, sizing, icons and visible interaction states. Extract reusable theme tokens; do not merely create “CRM-inspired” components with generic defaults. Capture the reference at the start so comparisons remain stable.

- Reimplement reusable components from the observed UI, not screenshots or an embedded copy of the reference site.
- Use typed props, realistic sample data and functional interactions. Include applicable hover, focus, selected and disabled states.
- Show side-by-side reference/recreation screenshots at comparable sizes; explain remaining visual differences.
- Do not rebuild the whole CRM. Recreate its component theme, not its business features.

3. Public catalogue experience

Frontend UX reference: [Astryx by Meta](#) — also explore its [component catalogue](#).

Use this reference for how developers discover components, navigate documentation, inspect previews and access integration instructions. Recreate that kind of catalogue experience using your own identity and the **Sales CRM component theme from Section 2**. Do not substitute Astryx's component styling for the CRM theme or imply affiliation with Meta. Its full feature set and component count are not required.

Provide a clear introduction/get-started page, searchable component navigation and a dedicated page per published component.

Each component page must include the following for authorised users; premium locked states follow Section 5:

- A working preview, useful variants/states and concise props/usage documentation.

- **Copy code:** actual source and example usage, including required theme/style files, imports and dependency instructions.
- **Copy install command:** a working `npm` command that adds the component and its required files to a separate consumer project.
- **Copy agent prompt:** a specific prompt telling an AI coding agent how to add this component, preserve the theme, satisfy dependencies and verify the integration.

Use readable code blocks, clear preview/code controls, copy-success/error feedback and a usable narrow-screen layout. This is a component exploration/documentation experience, not a sales dashboard or static marketing page.

Support **one documented React + TypeScript consumer setup**. Publishing to npm is not required: a publicly downloadable packaged CLI or compatible existing installer is acceptable. The displayed `npm` command must work on the reviewer's machine using the deployed component source, without candidate-local files, localhost services or cloning the entire catalogue. Document prerequisite setup.

4. Admin and publishing

Protect the admin app and write APIs with server-verified access. One administrator configured through environment variables is sufficient; no general-purpose roles system is needed. Customer sign-in is required for premium access, as described below. Never expose the admin secret in frontend code.

The admin must be able to:

1. List components and create/edit a draft.
2. Upload component source, supporting files and preview/example data through a documented format. A constrained JSON bundle is acceptable; building a code editor is unnecessary.
3. Set name, unique slug, description, category, version, free/premium access level, props/usage and declared dependencies.
4. Validate and preview the draft, then publish or unpublish it.
5. View customer accounts and grant or revoke premium access. A customer cannot change their own access level; premium status does not grant admin permissions.

Store records and files persistently. After publication, the new component must appear with a functioning preview and all three integration options for authorised users **without manually editing catalogue source, adding a per-component import or redeploying the frontend**. A normal page refresh is acceptable.

Drafts must stay private. Unpublishing must remove the component from public listings, direct detail routes and new installation/source requests. It does not need to revoke files already copied or installed.

5. Business requirement: premium components

Some components are free; others require an active premium account. Include examples of both, selected through your own reference analysis.

Visitor/account	Published free components	Published premium components
Signed out	Preview, copy, install and agent prompt	Public description/static thumbnail only; sign-in/access message
Signed in without premium	Full access	Locked; explain that premium access is required
Signed in with active premium	Full access	Full access

- - **Account flow:** provide working customer sign-in/sign-out and a visible free/premium account status. Seed a free and a premium test account for review. No public signup or password-recovery flow is required.
 - **Access model:** for this assignment, admins grant/revoke premium access; do not build real payments, prices, subscriptions or invoices. The locked UI should explain how access is obtained, not display a fake checkout.
 - **Backend enforcement:** check current access on every protected preview, source, download, installation and agent-prompt request, including direct URLs. Protect supporting files and dependencies that contain premium code. Hiding buttons is insufficient; revocation must block subsequent protected requests even if the customer remains signed in.
 - **Installer/agent access:** provide a documented authenticated path for premium installation and agent integration. Do not embed credentials in copied prompts/commands, public files or logs. Signed-out/free requests must fail clearly rather than silently install premium content.
 - **No source leakage:** keep runtime premium uploads out of public bundles, unrestricted storage/cache responses and the public Git repository. Use static thumbnails for locked previews. Public demo fixtures are acceptable as examples, but prove protection using a newly uploaded premium component not present in the repository.

Revocation prevents future retrieval; it cannot remove code someone already copied or installed. Document that limitation. Component publication and premium access are separate checks: premium customers must never retrieve drafts or unpublished content.

6. Core engineering expectations

- **Production readiness:** the deployed apps and integrations must work with persistent data, protected admin access, HTTPS, safe configuration and useful error handling/logging. Verify the deployed release and document recovery steps. A successful build alone is not evidence that the product works. This expectation applies to the stated scope, not unrequested enterprise features.
- **One consistent published version:** preview, copyable code, installer and agent prompt must reference the same component content and dependencies. Do not maintain unrelated hardcoded copies.
- **Safe uploads/previews:** validate required fields, supported file types, sizes and dependencies. Do not execute uploaded code in the backend or authenticated admin context. Use an isolated preview environment without access to app credentials; a documented restricted component format is acceptable.
- **Safe installation:** keep writes inside the selected consumer directory, reject unsafe paths and do not silently overwrite existing files or run component-supplied shell commands.
- **Usability:** keyboard-operable controls, labels, visible focus, and loading/empty/error states. Check interactions as well as appearance.

No multi-framework support, marketplace payments, public uploads, npm publication or general-purpose online code-execution platform is required.

7. Clean code and development standards

Apply these to all submitted code, including AI-generated code:

- **Clean code:** meaningful names, focused functions/components, consistent structure and no dead code. Separate presentation, publishing rules, storage and installation logic.
- **SOLID:** clear responsibilities, small contracts and testable dependencies. Use the principles where useful; do not add classes or architectural layers merely to demonstrate them.
- **DRY — Don't Repeat Yourself:** reuse theme tokens, shared behaviour and validation rules instead of maintaining divergent copies. Frontend validation improves usability; backend validation remains mandatory.
- **KISS / YAGNI:** choose the simplest maintainable solution for the stated scope. Avoid speculative features, premature optimisation and unnecessary abstractions.

- **Strict typing:** enable TypeScript `strict: true`; type component props, API contracts and registry data. Avoid unjustified `any` keyword, unsafe assertions and ignored type errors. Document narrow exceptions instead of disabling checks globally; use equivalent checks for a backend in another language.
- **Safe boundaries:** validate uploaded files and API inputs at runtime, use parameterised queries/safe ORM operations, handle errors clearly and keep secrets out of code and logs. Static types are not runtime validation.
- **Quality checks:** configure and run formatting, linting, type checks and meaningful tests. Include commands and actual results in the README; do not suppress failures to make checks appear successful.

Be ready to explain two concrete code-quality decisions and one abstraction you intentionally avoided. We assess maintainability in the implementation, not definitions memorised or patterns counted.

8. Demonstrate and test

Show this complete journey **on the deployed apps**, not just locally:

Upload an additional reference-derived component not already in the catalogue through admin → preview → publish → discover it publicly → use all three integration options.

Also demonstrate **publish a premium component → observe it locked as a free customer → admin grants premium → customer accesses preview/code/install/prompt → admin revokes premium → subsequent protected requests are denied**. Free components must remain usable throughout.

Verify copy/paste and CLI installation independently in clean consumer projects; both must build and render without catalogue-specific imports or missing theme files. Use the copied agent prompt with your AI tool in a clean consumer setup and record the result and any corrections.

Automate checks for:

1. Unauthorised admin writes and public access to drafts/unpublished components.
2. Valid publication and invalid-upload rejection.
3. Published metadata/source consistency and installer success.
4. Unsafe install paths and existing-file overwrite handling.
5. Free/premium access through direct API/source requests and the installer, including revoked access and customer attempts to grant themselves premium or perform admin actions.

Also record component interaction, keyboard/mobile, visual-comparison and consumer-build checks. Verify deployed admin authentication, publication/unpublication and persistence after an application restart or redeployment. Record preview isolation checks and known limitations; do not describe an ordinary unsandboxed iframe as secure isolation.

9. Final deliverables

Required final deliverables:

1. **Public Git repository link** containing the complete project: both apps, backend, installer/configuration, lockfiles, database/storage setup, tests and consumer examples. Identify the deployed commit.
2. **Deployed public frontend URL** with working free/premium catalogue states, customer sign-in and all three access-controlled integration options.
3. **Deployed admin dashboard URL** with working component publishing and premium-access management. Share admin, free-customer and premium-customer test credentials privately through the interview submission channel, never in the public repository, README or frontend bundle.
4. **An `answers.md` file in the repository** answering every question in Section 10. These answers are mandatory, alongside the working project.

The full project must be functional end to end. Deploy the backend, database and file storage needed by both apps; do not rely on services running on your laptop. Local demos, screenshots, static placeholders and non-working buttons are not substitutes. Keep the deployment available throughout the review. Free hosting is acceptable; a custom domain is not required.

Include a short README containing:

- The public links, local setup/build/check commands, deployment instructions and environment-variable names with a secret-free example. Never commit secrets or real customer data.
- Reference/recreation screenshots, actual test/build and deployed-flow results, time spent and known gaps. Plan deployment within the eight-hour timebox; flag blockers early. An unfinished core flow must be reported as incomplete, not presented as a completed submission.
- AI tool used, one representative prompt and one suggestion/assumption you verified or corrected, with code/test evidence. No full chat history is required.
- Release checks and a brief recovery plan: how you would diagnose a failed deployment, restore service and protect stored component data.
- Premium account setup, grant/revoke instructions, authenticated installer/agent usage and actual access-control test results. Do not include real payment or customer data.

Once finished with the assignment click the “Mark Done” in the Email and Share all the relevant links in the form.

10. Required written answers

Answer all seven in [answers.md](#), **2–4 sentences each**, adding code, test or screenshot references where useful. Explain decisions you personally made during development: what you chose, an alternative you considered, why you accepted or changed an AI suggestion where relevant, and how you checked the outcome. AI can help you draft, but generic generated explanations are insufficient: answers must match your actual implementation. Clearly distinguish implemented safeguards from future improvements; never invent verification results.

1. **Reference analysis:** how did you identify reusable components, variants and shared theme tokens? Explain one boundary you chose and how you verified the recreation against the reference.
2. **Architecture and clean code:** why did you choose this stack and separation of responsibilities? Show one practical SOLID/DRY decision and one abstraction or feature you avoided under KISS/YAGNI.
3. **Publishing consistency:** how do preview, copied code, installation and agent instructions stay on the same published version? What happens when an update fails or a component is unpublished?
4. **Security:** what could go wrong when previewing uploaded code, calling admin APIs or installing files into another project? Which protections did you implement and test, and what limitations remain?
5. **AI ownership:** which AI suggestion or assumption did you challenge, and what evidence supported your conclusion? Show how you checked that copied code, the install command and the agent prompt actually worked in consumer projects—not just inside the catalogue.
6. **Production ownership:** what checks convinced you the deployed project was ready? If a newly published component breaks after release, what would you inspect first, how would you restore service without losing data, and what would you communicate to the team?
7. **Premium access:** how did you model account access separately from component publication and admin permissions? Show how a free or revoked customer is blocked from premium code through previews, direct URLs, CLI and agent integration, and explain what revocation cannot undo.

11. Follow-up interview

The next interview will be based on **the assignment you built, the decisions you took and your application of the principles in Section 7**. Be ready to:

- Walk through your actual code and explain your component, architecture, publishing, premium-access, security and deployment choices—including alternatives you rejected.
- Show where you applied **clean code, SOLID, DRY, KISS, YAGNI, strict typing and testing**, and why those choices helped. Definitions alone are not enough; useful simple code is better than unnecessary patterns.

- Explain what AI produced, what you reviewed or changed, and the evidence that it works. You may be asked to debug an issue or make and verify a small change; AI and documentation remain allowed.

We assess practical understanding and ownership appropriate to your **experience**, not memorised terminology or how much code you typed unaided. Extra features cannot compensate for static previews, fake install commands or a catalogue that cannot display newly uploaded components.